



SHASU VATHANAN

SHASU VATHANAN - GEN AI - PRODUCT MANAGER

GEN AI ARCHITECT JOURNEY · PHASE-004

Streamlit

From Python Script to Interactive MVP

Document	A practical, step-by-step beginner's guide
Level	Beginner - no web development experience required
Tools	Python 3 · Streamlit · Windows Command Prompt
Phase	Phase-004 - Interfaces and APIs
Folder	Phase-004-FastAPI-And-Streamlit
You will build	A working Calculator MVP that runs in the browser

Shasu Vathanan - GEN AI - Product Manager

SHASUVATHANAN.COM · [linkedin.com/in/shasuvathanan](https://www.linkedin.com/in/shasuvathanan)



Contents

This guide moves in one direction: understand the tool, prepare the environment, build a working app, then improve it with feedback. Read the sections in order the first time through.

1	What is Streamlit?
2	What is an MVP?
3	Basic Streamlit app - the fastest path
4	Prepare the project
5	Create and run the first application
6	Build a simple Calculator MVP
7	Understand the Calculator MVP
8	Useful Streamlit building blocks
9	Streamlit files and project layout
10	Common problems and how to fix them
11	Final workflow

How to use this guide

Every command in this document is written for **Windows Command Prompt**, including the terminal inside Visual Studio Code when its profile is set to Command Prompt. Where macOS and Linux differ, the alternative is shown directly beneath.

1. What is Streamlit?

Streamlit is a free, open-source Python framework for building interactive web applications.

You write Python, and Streamlit creates the web interface. For a basic application you do not need to write separate HTML, CSS, or JavaScript. It is commonly used for:

- quick product prototypes and Minimum Viable Products (MVPs);
- data dashboards and charts;
- calculators and small business tools; and
- machine-learning or Generative AI demonstrations.

A simple way to understand a framework

Think of Streamlit as a set of ready-made interface blocks. When you need a heading, input, button, table, or chart, you call a Streamlit function such as `st.title()`, `st.number_input()`, or `st.button()`. The framework handles how those elements appear and respond in the browser. That lets you focus on the application's purpose and logic.



2. What is an MVP?

MVP means Minimum Viable Product - the smallest working version of an idea that still delivers its main value.

An MVP gives users the core benefit and lets you collect real feedback before building the full product. A complete calculator product may eventually include scientific operations, calculation history, user accounts, themes, and cloud storage. Its MVP can contain only:

1. two number inputs;
2. one choice of operation;
3. a **Calculate** button; and
4. the result.

That small version is enough to test whether the main calculation flow works and whether users understand it. Streamlit suits MVPs because the interface can be built, shown, tested, and improved quickly.

Product principle

Build the smallest useful flow, test it with users, learn from the feedback, and then add the next most valuable feature.

3. Basic Streamlit app - the fastest path

Open Command Prompt in the folder where you want to create the application, then follow this sequence.

Step	Command	Purpose
1. Create an environment	<code>python -m venv venv</code>	Creates an isolated Python environment named venv .
2. Activate it	<code>venv\Scripts\activate</code>	Activates the environment. The prompt then shows (venv) .
3. Install Streamlit	<code>python -m pip install streamlit</code>	Installs Streamlit inside the active environment. Run once per environment.
4. Check the install	<code>streamlit --version</code>	Confirms Streamlit is installed and prints its version.
5. Run an app	<code>streamlit run app.py</code>	Starts the Streamlit file named app.py .

The required commands

CMD

```
python -m venv venv
venv\Scripts\activate
streamlit --version
streamlit run app.py
```

To leave the environment when you have finished:

CMD

```
deactivate
```



The basic application file

Item	Details
File name	Streamlit_Basic.py
Command to run	streamlit run Streamlit_Basic.py
Expected browser address	http://localhost:8501
Stop the application	Press Ctrl + C in Command Prompt

Create or open **Streamlit_Basic.py** and use this code:

```
PYTHON
# Streamlit Basic
import streamlit as st

st.title("Streamlit Basic")
st.write("This is a basic Streamlit app")
```

Line-by-line explanation

Code	What it does
# Streamlit Basic	A Python comment that identifies the example.
import streamlit as st	Imports Streamlit and gives it the standard short name st .
st.title("Streamlit Basic")	Displays the main heading in the browser.
st.write("This is a basic Streamlit app")	Displays the supporting text under the heading.

Make sure the environment is active and Command Prompt is in the folder containing the file, then run it. Streamlit opens the application in your default browser. If it does not open automatically, copy the **Local URL** shown in Command Prompt and open it manually.

4. Prepare the project

Step 1 - Check Python

```
CMD
py --version
py -m pip --version
```

If **py** is unavailable but **python** works, replace **py** with **python** in every command.

Step 2 - Create and open a project folder

```
CMD
mkdir streamlit_mvp
cd streamlit_mvp
```



Step 3 - Create a virtual environment

```
CMD
py -m venv .venv
```

A virtual environment keeps this project's packages separate from every other Python project on the machine.

Step 4 - Activate it correctly

```
CMD
.venv\Scripts\activate.bat
```

The prompt should now begin with **(.venv)**. On macOS or Linux the equivalent command is:

```
BASH
source .venv/bin/activate
```

Step 5 - Install and verify Streamlit

```
CMD
python -m pip install streamlit
python -m streamlit --version
```

After activation, using **python -m pip** guarantees that Streamlit is installed into the active virtual environment rather than a global one.

Step 6 - Confirm Streamlit with its built-in demo

```
CMD
python -m streamlit hello
```

A browser should open with Streamlit's example application. Return to the terminal and press **Ctrl + C** to stop it.

5. Create and run the first application

Create a file named **app.py** and add:

```
PYTHON
import streamlit as st

st.title("My First Streamlit App")
st.write("Hello! Welcome to Streamlit.")
```

- `import streamlit as st` imports Streamlit under its standard short name.
- `st.title()` creates the main page heading.
- `st.write()` displays text or other Python values.

Run and update the application

```
CMD
python -m streamlit run app.py
```

The general form of the command is `python -m streamlit run <file-name>.py`. Streamlit starts a local web server and normally opens the app in a browser at **<http://localhost:8501>**.



Important

Do not start a Streamlit interface with **python app.py**. The **streamlit run** command is what manages the web server and the interactive reruns - without it you get plain script output and no interface.

The development loop

1. Edit the Python file.
2. Save it with **Ctrl + S**.
3. Return to the browser.
4. Select **Rerun** if Streamlit asks, or enable **Always rerun**.
5. Review the result and continue improving it.

Streamlit executes the script from top to bottom after every widget interaction, using the current widget values during the new run.

6. Build a simple Calculator MVP

Replace the contents of **app.py** with this complete example:

PYTHON

```
import streamlit as st

st.set_page_config(page_title="Calculator MVP", page_icon="CALC")

st.title("Calculator MVP")
st.write("Enter two numbers, select an operation, and calculate the result.")

number_1 = st.number_input("First number", value=0.0)
number_2 = st.number_input("Second number", value=0.0)
operation = st.selectbox(
    "Operation",
    ["Add", "Subtract", "Multiply", "Divide"],
)

if st.button("Calculate", type="primary"):
    if operation == "Add":
        result = number_1 + number_2
    elif operation == "Subtract":
        result = number_1 - number_2
    elif operation == "Multiply":
        result = number_1 * number_2
    elif number_2 == 0:
        result = None
        st.error("Division by zero is not allowed.")
    else:
        result = number_1 / number_2

    if result is not None:
        st.success(f"Result: {result:g}")
```

Save the file. The running application reruns automatically, or after you select **Rerun**.



7. Understand the Calculator MVP

Page setup and content

`st.set_page_config()` sets browser-page information and must be the first Streamlit command in the file. `st.title()` and `st.write()` explain the page's purpose.

Inputs

`st.number_input()` returns a numeric value. Each call uses a different label so users know which value to enter.

`st.selectbox()` limits the operation to four valid choices, which removes a whole class of invalid input.

Action and decision logic

`st.button()` returns **True** when clicked. The **if** and **elif** statements then apply the selected operation. This logic is ordinary Python; Streamlit only supplies the interface around it.

Validation and output

Division by zero is checked before the division takes place. `st.error()` explains invalid input, while `st.success()` presents a valid result. A good MVP covers the main success path and at least one important failure path.

What to test before sharing

Test	Input	Expected result
Addition	10, 5, Add	15
Subtraction	10, 5, Subtract	5
Multiplication	10, 5, Multiply	50
Division	10, 5, Divide	2
Invalid division	10, 0, Divide	A clear error message, no crash

Then ask a test user to complete a calculation without any instructions. Observe whether the labels, operation choice, button, and result are easy to understand. That feedback determines what to improve next.



8. Useful Streamlit building blocks

Command	Purpose
<code>st.title("Text")</code>	Main page title
<code>st.write(value)</code>	Text, values, and many Python objects
<code>st.text_input("Label")</code>	Single-line text input
<code>st.number_input("Label")</code>	Numeric input
<code>st.selectbox("Label", options)</code>	Drop-down choice
<code>st.button("Label")</code>	Clickable action
<code>st.dataframe(data)</code>	Interactive data table
<code>st.line_chart(data)</code>	Quick line chart
<code>st.success("Text")</code>	Success feedback
<code>st.warning("Text")</code>	Warning feedback
<code>st.error("Text")</code>	Error feedback

These same building blocks later support dashboards, chart applications, and Generative AI model demonstrations.

9. Streamlit files and project layout

Place the application in a normal Python `.py` file and run it from a terminal. Do not paste the app into a Jupyter `.ipynb` cell and expect the same browser-app workflow.

```
TEXT
streamlit_mvp/
  .env/
  app.py
  requirements.txt
```

Create a small dependency file manually containing one line:

```
TEXT
streamlit
```

Install it on another computer with:

```
CMD
python -m pip install -r requirements.txt
```

Never share `.env`

A virtual environment is machine-specific. Commit `requirements.txt` instead, and let the next person recreate the environment from it.



10. Common problems and how to fix them

Problem	Cause	Fix
Streamlit is not found	The environment is not active	Confirm <code>(.venv)</code> appears in the prompt, then run <code>python -m streamlit run app.py</code>
The module is missing	Streamlit was installed elsewhere	Run <code>python -m pip install streamlit</code> inside the active environment
The application file is not found	Wrong working folder	Run <code>dir</code> , confirm <code>app.py</code> is listed, and <code>cd</code> into the correct folder
The browser does not open	No default browser handoff	Copy the Local URL printed in the terminal and open it manually
A code change does not appear	The file was not saved or the script errored	Save the file, select Rerun , and check the terminal for a Python error
Port 8501 is already in use	A previous server is still running	Stop it with <code>Ctrl + C</code> , or run <code>python -m streamlit run app.py --server.port 8502</code>

11. Final workflow

1. Open Command Prompt in the project folder.
2. Activate with `.venv\Scripts\activate.bat`.
3. Install Streamlit with `python -m pip install streamlit`.
4. Write the interface in `app.py`.
5. Run it with `python -m streamlit run app.py`.
6. Test the main user flow and the important errors.
7. Collect user feedback before expanding the MVP.
8. Stop the server with `Ctrl + C`.
9. Deactivate the environment with `deactivate` when finished.

Possible next improvements

After the MVP is validated, add one valuable feature at a time - for example calculation history, more operations, or a downloadable result. Avoid adding features until the main flow is clear and useful.

Next

Apply this workflow in the Streamlit Student Grade App, the Phase-004-FastAPI-And-Streamlit assignment build.

Shasu Vathanan - GEN AI - Product Manager
SHASUVATHANAN.COM · linkedin.com/in/shasuvathanan